Algoritmo de Bernstein–Vazirani

Genesis Silesky

21 de octubre de 2026

1. Algoritmo de Bernstein–Vazirani

1.1. Objetivo

Descubrir un número binario oculto de n bits con una sola consulta, en vez de las n consultas que necesita un algoritmo clásico, y ver que la ventaja viene de la superposición y no del entrelazamiento.

1.2. Conceptos previos

Concepto previo: puertas H sobre varios qubits y producto punto módulo 2

Ver capítulos 05a y 15. El *producto punto* de dos cadenas de bits es $a \cdot x = a_1x_1 + a_2x_2 + \cdots + a_nx_n$, y "módulo 2" significa quedarse con el resto de dividir entre 2 (la paridad).

1.3. Fundamento teórico

El algoritmo fue propuesto por Bernstein y Vazirani en 1993. Muestra la ventaja cuántica en términos de número de consultas.

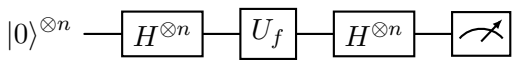
El problema. Una base de datos guarda un número oculto $a \in \{0,1\}^n$. Se puede hacer una consulta eligiendo un $x \in \{0,1\}^n$, y la respuesta es

$$f(x) = a \cdot x \pmod 2 = a_1x_1 \oplus a_2x_2 \oplus \cdots \oplus a_nx_n, \tag{1}$$

un solo bit (0 o 1). El objetivo es hallar a.

Solución clásica. Se consulta con $x = 100\ldots0$, $010\ldots0$, …, $000\ldots1$: la consulta i da el bit a_i. Hacen falta n **consultas**. Por ejemplo, para $a = 1011$ se necesitan cuatro.

Solución cuántica. Se prepara una superposición uniforme de n qubits y se usa como entrada de la función, de modo que todas las consultas se hacen en paralelo. El circuito de este ejemplo es una variante de Phys. Rev. A **64**, 042306, y no usa qubits auxiliares:



con el oráculo $U_f\,|x\rangle = (-1)^{a\cdot x}\,|x\rangle$.

Por qué funciona. Una H sobre un qubit da $H|a\rangle = \frac{1}{\sqrt{2}}\sum_{x\in\{0,1\}}(-1)^{ax}|x\rangle$. Para n qubits:

$$H^{\otimes n}|a\rangle = \frac{1}{\sqrt{2^n}}\sum_{x\in\{0,1\}^n}(-1)^{a\cdot x}|x\rangle\,, \qquad H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}}\sum_{x}|x\rangle\,. \tag{2}$$

Es decir, U_f convierte la superposición uniforme en $H^{\otimes n}|a\rangle$. Como $H^{\otimes n}H^{\otimes n} = I$, la H final lo lleva de vuelta a $|a\rangle$:

$$|a\rangle = H^{\otimes n}\,U_f\,H^{\otimes n}\,|0\rangle^{\otimes n}\,. \tag{3}$$

Al medir los n qubits se lee directamente a. **Una sola consulta**, es decir, complejidad $O(1)$ frente a $O(n)$ clásica.

Sin entrelazamiento. El oráculo se arma como un producto de n puertas de un solo qubit: $U_i = I$ si $a_i = 0$ y $U_i = Z$ si $a_i = 1$. Todas las puertas del circuito son de un qubit y el estado inicial no está entrelazado, así que el estado nunca lo está. La aceleración viene de la **superposición**, no del entrelazamiento.

1.4. Código (completamente comentado)

```python
# %% [markdown]
# # 16 - Algoritmo de Bernstein-Vazirani: adivinar un numero oculto con una consulta

# %% Celda 1: importaciones
import numpy as np
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector, entropy, partial_trace

sim = AerSimulator()

# %% Celda 2: el problema
# Hay un numero oculto de n bits, a = a1 a2 ... an. Solo se puede hacer una pregunta:
# se elige un x de n bits y se recibe f(x) = a . x (mod 2), es decir, la suma (mod 2)
# de los bits donde a y x valen 1. La respuesta es un solo bit (0 o 1).
a_secreto = "1011"          # lo "desconoce" el algoritmo; a1 es el primer bit de la izquierda
n = len(a_secreto)
a = [int(b) for b in a_secreto]


def f(x):
    """La caja negra: devuelve a . x (mod 2)."""
    return sum(ai * xi for ai, xi in zip(a, x)) % 2


# %% Celda 3: solucion clasica, n consultas
# Se pregunta con x = 1000, 0100, 0010, 0001: la respuesta i-esima da el bit a_i.
encontrado = ""
for i in range(n):
    x = [1 if j == i else 0 for j in range(n)]
    encontrado += str(f(x))
    print(f"consulta {i + 1}: x = {''.join(map(str, x))} -> f(x) = {f(x)}")
print("Numero encontrado:", encontrado, f"({n} consultas)")

# %% Celda 4: el oraculo cuantico
# En la version de este ejemplo el oraculo actua asi:  Uf |x> = (-1)^(a . x) |x>.
# Se arma como un producto de puertas de UN solo qubit: una Z en cada qubit donde a_i = 1
# y nada donde a_i = 0. (Z cambia el signo de |1>.)
# Qiskit numera el qubit 0 a la DERECHA al escribir cadenas; aqui el qubit i-1 guarda a_i.
def oraculo(a_bits):
    qc = QuantumCircuit(len(a_bits))
```

```
    for i, ai in enumerate(a_bits):
        if ai == 1:
            qc.z(i)
    return qc


# %% Celda 5: el circuito de Bernstein-Vazirani
#   |0...0> -> H en todos -> Uf (una sola consulta) -> H en todos -> medir.
# La H final deshace la superposicion y deja en los qubits justo el numero a.
def bernstein_vazirani(a_bits):
    n = len(a_bits)
    qc = QuantumCircuit(n, n)
    qc.h(range(n))
    qc.compose(oraculo(a_bits), inplace=True)
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc


# En Colab o Jupyter el circuito se dibuja solo; en terminal no se muestra.
bernstein_vazirani(a).draw()

# %% Celda 6: ejecutar, con una sola consulta
cuentas = sim.run(bernstein_vazirani(a), shots=1000).result().get_counts()
# Qiskit escribe el qubit 0 a la derecha, asi que se invierte la cadena para leer a1...an.
medido = max(cuentas, key=cuentas.get)[::-1]
print("Mediciones:", cuentas)
print("Numero oculto:", a_secreto, "| medido:", medido, "| 1 consulta en vez de", n)

# %% Celda 7: sin entrelazamiento
# Todas las puertas son de un solo qubit y el estado inicial no esta entrelazado, asi
# que el estado nunca lo esta: la ventaja viene de la superposicion, no del
# entrelazamiento. Se comprueba con la entropia de un qubit (0 = no entrelazado).
sin_medir = QuantumCircuit(n)
sin_medir.h(range(n))
sin_medir.compose(oraculo(a), inplace=True)
sin_medir.h(range(n))
sv = Statevector(sin_medir)
for q in range(n):
    resto = [i for i in range(n) if i != q]
    print(f"entropia del qubit {q}: {float(entropy(partial_trace(sv, resto), base=2)):.3f}")
```

1.5. Explicación celda por celda

Celda 2 Define el número oculto $a = 1011$ y la caja negra $f(x) = a \cdot x$.

Celda 3 Solución clásica: cuatro consultas con $x = 1000, 0100, 0010, 0001$.

Celda 4 El oráculo: una Z en cada qubit donde $a_i = 1$.

Celda 5 El circuito $H^{\otimes n}$, U_f, $H^{\otimes n}$ y medición.

Celda 6 Lo ejecuta (1000 mediciones) e invierte la cadena, porque Qiskit escribe el qubit 0 a la derecha.

Celda 7 Calcula la entropía de cada qubit antes de medir para comprobar que no hay entrelazamiento.

1.6. Salida obtenida

```
consulta 1: x = 1000 -> f(x) = 1
consulta 2: x = 0100 -> f(x) = 0
consulta 3: x = 0010 -> f(x) = 1
consulta 4: x = 0001 -> f(x) = 1
Numero encontrado: 1011 (4 consultas)
Mediciones: {'1101': 1000}
Numero oculto: 1011 | medido: 1011 | 1 consulta en vez de 4
entropia del qubit 0: 0.000
entropia del qubit 1: 0.000
entropia del qubit 2: 0.000
entropia del qubit 3: 0.000
```

(El dibujo del circuito no se reproduce aquí.)

1.7. Interpretación

Las cuatro consultas clásicas dan $1, 0, 1, 1$, es decir, $a = 1011$. El circuito cuántico, con una sola consulta, mide siempre `1101` en 1000 mediciones; al invertir la cadena se lee `1011`, el número oculto. La entropía de cada qubit es 0: el estado nunca está entrelazado.

1.8. Ejercicios

1. Prueba con otro número oculto, por ejemplo $a = 0110$ y $a = 11111$.
2. ¿Qué pasa si $a = 0000$? Predícelo y compruébalo.
3. Cuenta cuántas consultas necesita el algoritmo clásico para $n = 20$ y cuántas el cuántico.

1.9. Errores comunes

- Olvidar que Qiskit imprime el qubit 0 a la derecha: la cadena medida está al revés respecto de $a_1 a_2 \dots a_n$.
- Confundir el número de *consultas* con el tiempo total: el algoritmo cuenta cuántas veces se usa la caja negra.